

Document Control Bot — System Architecture

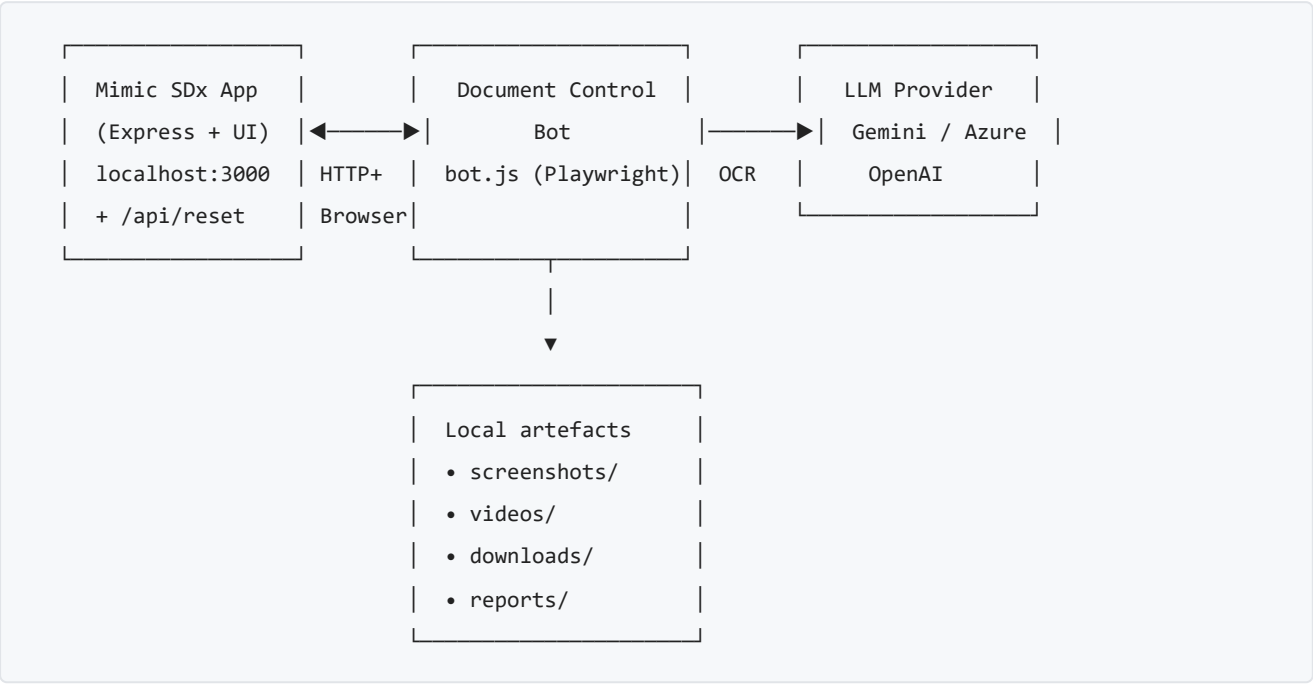
Component: Document Control Automation Bot (POC) **Entry point:** `hxgn-sdx-bot/bot/bot.js`
Runtime: Node.js + Playwright (Chromium), single process **Owner:** Document Control Automation
Date: 2026-04-16

1. Overview

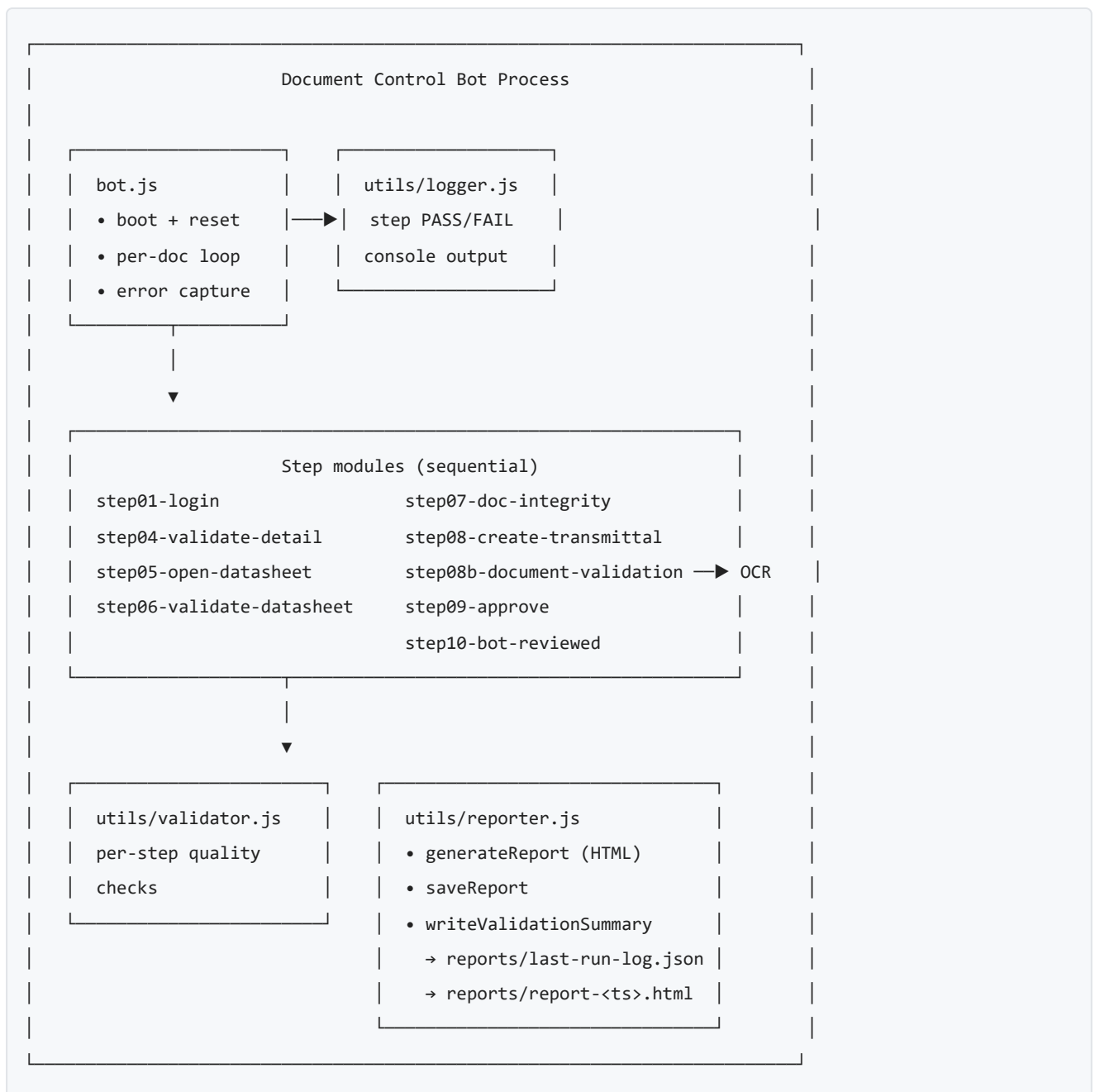
The bot is a **single-instance, synchronous UI automation script**. It logs into the SDx mimic portal, iterates over every document in the "Review Submittal" state on the To Do list, and runs a fixed sequence of steps per document. Per-document failures are caught, archived, and reported — the loop continues to the next document.

There is no polling daemon, no database, no distributed lock, and no multi-instance coordination. The OCR validation subsystem invoked at Step 8b is documented separately in `OCR_ARCHITECTURE.md`.

2. Context



3. Components



4. Per-document processing sequence

For each row in To Do list with status "Review Submittal":

Step 1 (once)	step01-login	Login + land on To Do screen
Step 2	claim button click	Acquire the document in the UI
Step 3	step04-validate-detail	Detail-screen field checks
Step 4	step05-open-datasheet	Open the datasheet panel
Step 5	step06-validate-datasheet	Datasheet field validation
Step 6	step07-doc-integrity	Download PDF + integrity checks
Step 7	step08-create-transmittal	Create incoming transmittal (TR-CP2-XXXX)
Step 8 (8b)	step08b-document-validation	AI OCR validation —► see OCR_ARCHITECTURE.md
Step 9	step09-approve	Approve or reject the submittal
Step 10	step10-bot-reviewed	Mark "Bot Reviewed" in UI

After each step:

- full-page screenshot → screenshots/doc<NN>-<phase>.png
- optional STEP_DELAY pause for video pacing
- validator results pushed into docResults.steps[]

After all documents:

- generateReport(...) → reports/report-<timestamp>.html
- writeValidationSummary(...) → reports/last-run-log.json

5. Failure handling (what actually happens)

The bot uses a per-document `try/catch` in `bot.js`. There is **no retry, no checkpoint, no resume from prior state** — the policy is "fail fast, archive, move on".

Failure type	Behaviour
Step throws (any step)	Document marked <code>FAIL</code> , error message captured in <code>docResults.steps[]</code> , loop continues to next document.
Step returns <code>{status: 'FAIL'}</code> (non-fatal)	Document outcome set to <code>FAIL</code> after Step 10, but remaining steps still execute and artefacts are preserved.
Step 8b OCR returns FAIL	<code>bot.js</code> re-throws to short-circuit Steps 9–10; the failing PDF is archived.
PDF archive on failure	Source PDF copied from <code>mimic-app/backend/test-pdfs/</code> to <code>bot/downloads/<docNum>_REJECTED_<ISO-stamp>.pdf</code> .
Mimic app unreachable on <code>/api/reset</code>	Logged as a warning; bot continues anyway.
Process crash mid-document	No recovery. The next run starts fresh after <code>/api/reset</code> and re-processes from the mimic's seeded state.

OCR-layer transient errors (HTTP 429 from Gemini/Azure) are retried inside the OCR adapters with a fixed [20, 35, 50]s back-off — see `OCR_ARCHITECTURE.md` §5.

6. Environment configuration

Loaded from `hxgn-sdx-bot/.env` via `dotenv`.

Variable	Default	Purpose
<code>APP_URL</code>	<code>http://localhost:3000</code>	Mimic SDx portal base URL
<code>BOT_USERNAME</code>	<code>dc_bot</code>	Login username
<code>BOT_PASSWORD</code>	<code>BotPass2026!</code>	Login password
<code>HEADLESS</code>	<code>true</code>	Set <code>false</code> to watch the run
<code>SLOW_MO</code>	<code>0</code>	Playwright per-action delay (ms)
<code>STEP_DELAY</code>	<code>0</code>	Pause between steps (ms) — useful for video
<code>AI_PROVIDER</code>	<code>gemini</code>	OCR engine — <code>gemini</code> or <code>azure</code>
<code>GEMINI_API_KEY</code>	—	Required when <code>AI_PROVIDER=gemini</code>
<code>AZURE_OPENAI_API_KEY</code> / <code>_ENDPOINT</code> / <code>_DEPLOYMENT</code>	—	Required when <code>AI_PROVIDER=azure</code>

7. Outputs

Path	Contents
<code>bot/screenshots/</code>	<code>doc<NN>-<phase>.png</code> per document per step
<code>bot/videos/</code>	Playwright session recording (one <code>.webm</code> per run)
<code>bot/downloads/</code>	Downloaded submittal PDFs + <code>_REJECTED_*.pdf</code> archives + <code>_errors.pdf</code> annotated outputs
<code>bot/reports/last-run-log.json</code>	Machine-readable summary of the most recent run
<code>bot/reports/report-<ts>.html</code>	Per-run human-readable HTML report
<code>bot/reports/architecture-flow.html</code>	Static run-flow diagram

8. File map

Role	Path
Entry point / orchestrator	hxgn-sdx-bot/bot/bot.js
Step modules	hxgn-sdx-bot/bot/steps/step01-login.js ... step10-bot-reviewed.js (+ step08b-document-validation.js)
Per-step quality checks	hxgn-sdx-bot/bot/utils/validator.js
Logger	hxgn-sdx-bot/bot/utils/logger.js
Reporter	hxgn-sdx-bot/bot/utils/reporter.js
OCR adapters	hxgn-sdx-bot/bot/utils/gemini-ocr.js · azure-openai-ocr.js
Mimic SDx app	hxgn-sdx-bot/mimic-app/ (Express server + UI on port 3000)
Test PDF generator	hxgn-sdx-bot/mimic-app/backend/generate-test-pdfs.js

9. Roadmap to lower-environment & production rollout

The POC validates the end-to-end document journey on a single workstation. To promote the bot through DEV → SIT → UAT → PROD, the following capabilities are planned. Each item maps directly to a section of [SYSTEM_ARCHITECTURE_TARGET.md](#).

#	Capability	Target environment	Planned approach
1	Polling daemon / job queue	DEV → SIT	Convert <code>bot.js main()</code> into a long-running scheduler that polls SDx every 5 s for documents in <code>Review Submittal</code> . Wrap as a Node.js service (PM2 / systemd / Windows Service).
2	Database persistence	SIT onwards	Introduce PostgreSQL with <code>audit_log</code> , <code>document_locks</code> , <code>processing_checkpoint</code> , <code>retry_history</code> tables (schema in target doc §"Database Schema"). Replace in-memory <code>docResults[]</code> with DB writes after each step.
3	Distributed locking / multi-instance coordination	UAT → PROD	Use the <code>document_locks</code> row-level mutex with 30 s expiry so 2–3 bot instances can run behind a load balancer for HA. Staggered poll offsets (0, 1.66s, 3.33s) to avoid thundering herd.
4	Crash recovery / checkpoint resume	SIT onwards	Persist tier-completion state to <code>processing_checkpoint</code> after each step; on restart, resume the document from the last successful step rather than restarting the SDx workflow.
5	Bot-side /api/health endpoint	DEV → SIT	Add an Express health probe on the bot process exposing <code>{status, poll_active, documents_pending, last_run_at}</code> for ops dashboards and Kubernetes liveness/readiness.
6	API-down resilience	UAT → PROD	Direct-DB fallback path so transient SDx UI outages do not stall processing; bot reconciles back through the API once it returns.
7	Centralised logging & alerting	SIT onwards	Stream <code>logger.js</code> output to ELK / Splunk; raise alerts on stalled docs, lock contention, OCR 429 rate, and poll latency thresholds defined in target doc §"Monitoring & Alerting".
8	Secrets management	SIT onwards	Move <code>GEMINI_API_KEY</code> , <code>AZURE_OPENAI_API_KEY</code> , and SDx credentials from <code>.env</code> into Azure Key Vault / HashiCorp Vault and inject at runtime.
9	CI/CD & containerisation	DEV → PROD	Dockerise the bot + Playwright dependencies; deploy via Azure DevOps pipeline with environment-specific config and blue/green rollout for the polling fleet.

The current POC provides the validated business logic (Steps 1–10 + OCR Step 8b). The roadmap above is purely operational hardening — no functional re-write is required to reach production.